

Complete Smart Contract and UI Network Architecture

Diamond facets, enums, structs, variables, storage slots, functions, events, roles, subgraph entities, UI call paths, URL classes, retry behavior, and request-count formulas.

- Checked-in source snapshot and read-only live Base Sepolia topology are documented separately.
- All endpoint examples are redacted patterns. No private key, client secret, or environment value is included.
- Call counts distinguish logical operations, batched HTTP transport, retry attempts, timers, and user actions.

Scope, evidence, and notation

Most important finding

The source repository is not the source generation deployed at the active Base Sepolia Diamond. The report therefore has a Source layer and a Live/indexed layer.

Snapshot date: 2026-07-29 UTC. The worktrees contained pre-existing UI changes; commit IDs below identify the repository bases, while the analysis reflects the visible working tree. No build, test, deployment, transaction, or secret-value read was required.

Repository	Base commit	Evidence used
p2pflow-smart-contract	6823a7a	Solidity source, artifacts, deploy/upgrade scripts
p2pflow-subgraph	157207a	Active Base Sepolia manifest, ABI, schema, mappings
p2pflow-admin-ui	142fba4	Admin React/Vite client
p2pflow-merchant-ui	13264f1	Merchant React/Vite client
p2pflow-user-ui	3f64def	User React/Vite client

Count notation

- D = number of open disputes returned; N = applications; U = distinct merchant wallets; M = merchants; C = channels.
- Logical RPC count means JSON-RPC methods such as eth_call. One HTTP POST can carry many batched methods.
- Cold means no usable cache entry. Successful count assumes the first attempt succeeds. Persistent failure count includes configured retries.
- On-demand means triggered by a user action; recurring means timer/focus/remount can repeat it.

Architecture conclusions

Live deployment

Base Sepolia chain 84532; Diamond 0xF40AD901CCfB5E5EdC5162D6Ac7DDd5Ed5899F3A; active subgraph starts at block 44359818.

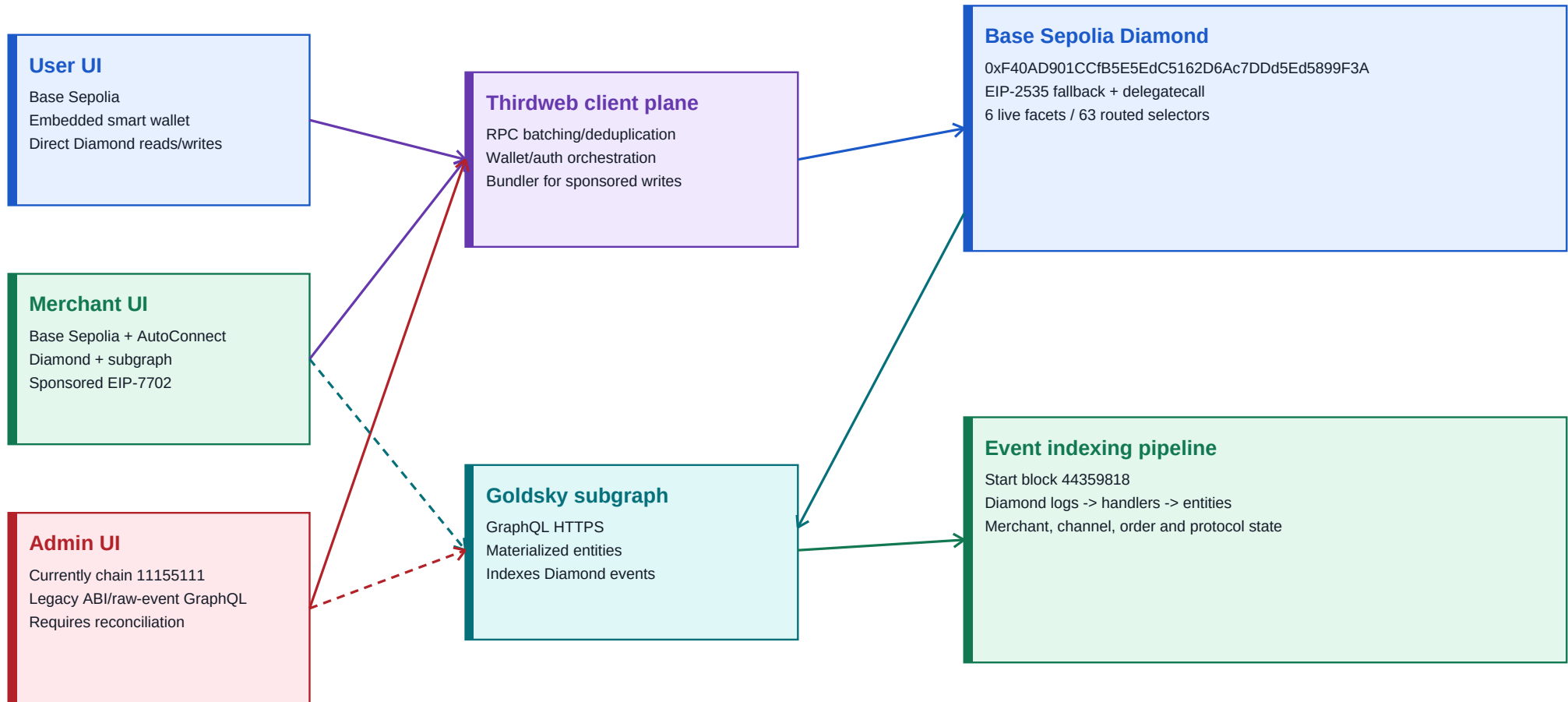
- Checked-in Solidity: five facets, 36 routed functions, exact AppStorage slots 0 through 8.
- Live Diamond: six facets, 63 routed functions. The active ABI also contains `init(...)`, but loupe does not route it.
- User and merchant clients target Base Sepolia; admin currently constructs Ethereum Sepolia chain 11155111 despite Base labels/configuration.
- Merchant guard cold-start success produces two distinct subgraph POSTs. Under persistent failure, default retry behavior can produce eight attempts total.
- User order details refetch every six seconds: approximately ten logical reads per minute after the initial read.
- The active admin ABI and raw-event subgraph queries are incompatible with the live selector surface/materialized schema.
- Frontend `VITE_*` variables are public bundle inputs. Names containing `SECRET` must not hold server secrets.

Upgrade stop condition

Do not diamondCut the live Base deployment from the checked-in source until its exact deployed source and storage layout are recovered.

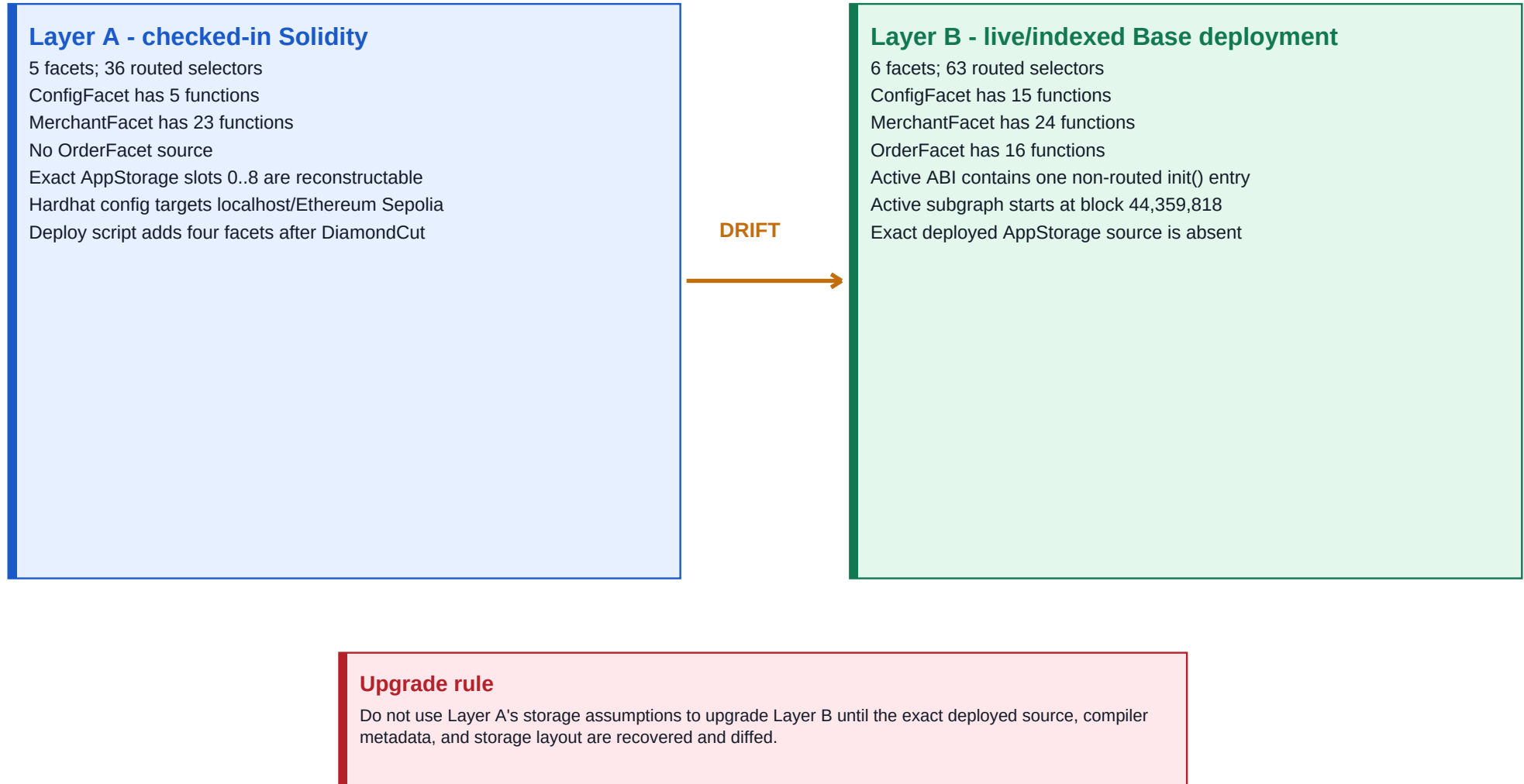
System context

Browser clients read/write the Diamond through Thirdweb/RPC and read indexed state through Goldsky GraphQL.



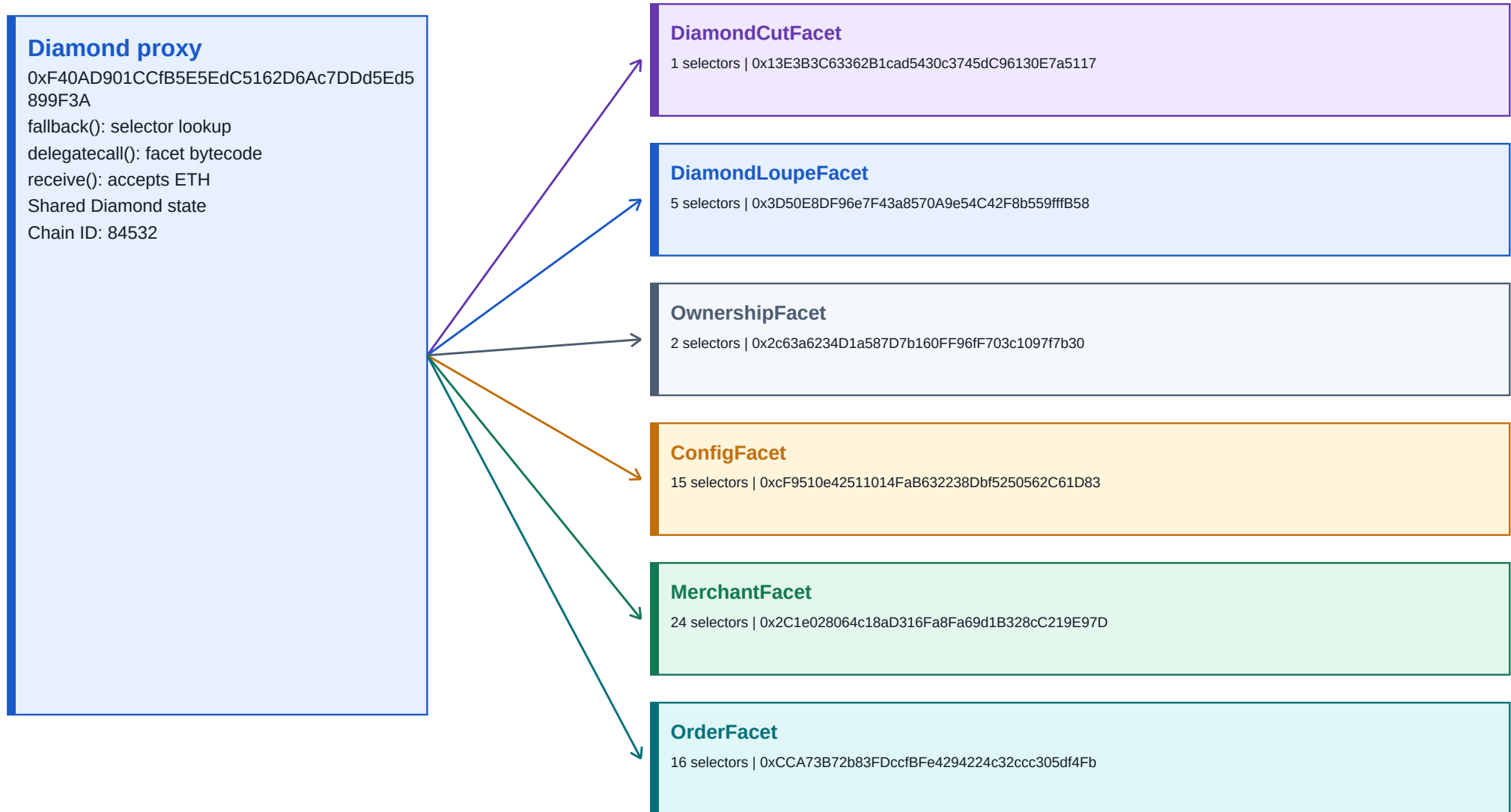
Two truth layers must not be merged

The repository source is authoritative for its storage layout; the live loupe and active ABI are authoritative for the deployed selector surface.



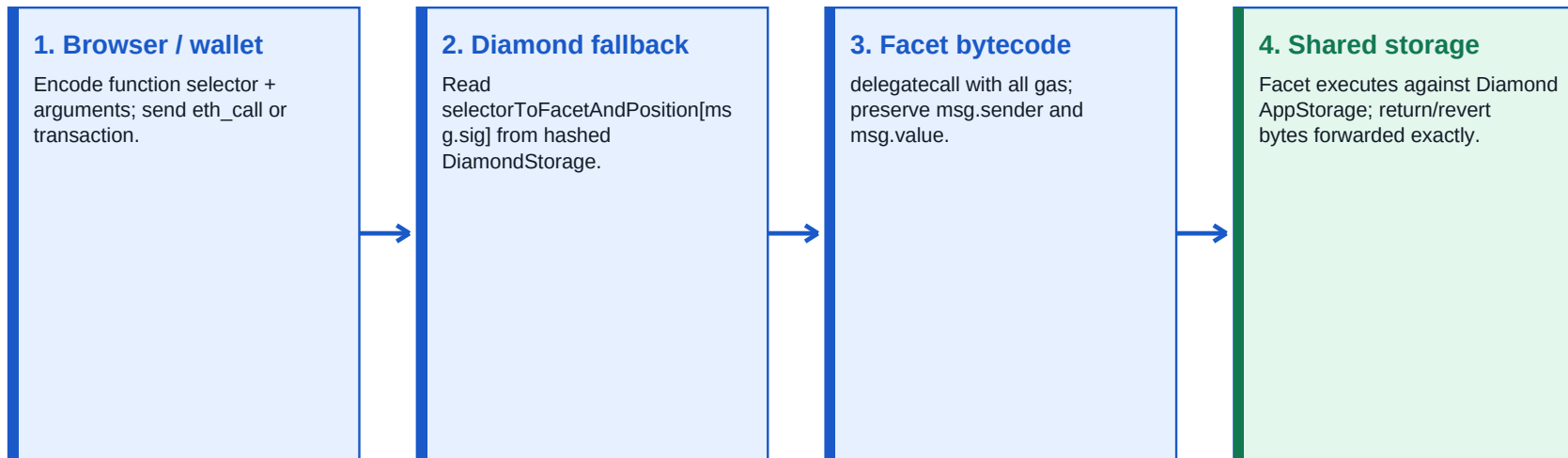
Live Base Sepolia Diamond

Read-only loupe inspection resolved six facet addresses and 63 selectors.



Runtime selector routing

Every integration must call the Diamond address, not a facet address.



Failure path

No route -> revert 'Diamond: Function does not exist'. Facet revert data is copied back unchanged. Direct facet calls observe the facet contract's own uninitialized storage.

Disjoint storage regions

Business AppStorage begins at slot zero; selector/ownership state begins at a fixed keccak256 slot.

AppStorage at slots 0..8

- 0 config.admin
- 1 config.usdcToken + paused
- 2 config.minMerchantStakeUsdc
- 3 config.initialized
- 4 merchants mapping seed
- 5 merchantList
- 6 channels mapping seed
- 7 duplicate guard mapping seed
- 8 reentrancy status

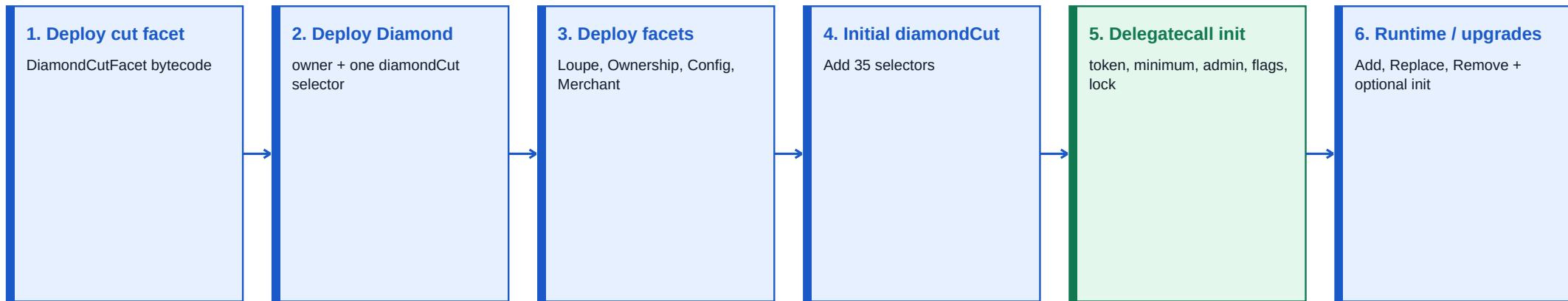
DISJOINT

DiamondStorage at P

P = keccak256('diamond.standard.diamond.storage')
P+0 selector -> facet/position mapping
P+1 facet -> selector array/position mapping
P+2 facetAddresses dynamic array
P+3 supportedInterfaces mapping
P+4 contractOwner
Region is intentionally disjoint from slot zero

Bootstrap, initialization, and upgrade

Owner-controlled diamondCut is the ultimate authority boundary.



Critical upgrade invariant

Preserve every existing AppStorage field's order and type; append only. The current upgrade script does not remove dropped selectors and sends no migration initializer.

Checked-in source component inventory

This section is exact for p2pflow-smart-contract at the inspected working-tree snapshot. It is not asserted to be the deployed Base implementation.

Component	Role	Key dependencies
Diamond	Permanent payable proxy; constructor installs diamondCut; fallback delegatecalls; receive accepts ETH	LibDiamond, IDiamondCut
DiamondCutFacet	Owner-only selector upgrade entrypoint	LibDiamond
DiamondLoupeFacet	Five routing/interface inspection views	LibDiamond, IDiamondLoupe, IERC165
OwnershipFacet	Read/transfer Diamond owner	LibDiamond, IERC173
ConfigFacet	Platform pause, minimum stake, admin transfer	Modifiers/AppStorage
MerchantFacet	Merchant registration, USDC custody, channels, admin actions	Modifiers, LibMerchants, SafeERC20
DiamondInit	One-time initialization delegatecall; not a facet in deploy script	Modifiers, LibDiamond
LibDiamond	Hashed routing storage; add/replace/remove algorithms	IDiamondCut
LibMerchants	Channel ID, bank normalization, digit helpers	Pure internal library
Modifiers	onlyAdmin, notPaused, shared nonReentrant lock	AppStorage s

Absent from source

No AdminFacet, OrderFacet, EscrowFacet, limits facet, or dedicated dispute facet exists in this Solidity tree.

Enums and numeric values

Enum	Member	Value	Meaning
MerchantAccountStatus	ACTIVE	0	Registered/operational account state
MerchantAccountStatus	INACTIVE	1	Unstake workflow state
MerchantAccountStatus	BLACKLISTED	2	Administrative terminal restriction; no unblacklist API
MerchantAccountStatus	DISPUTED	3	Administrative dispute restriction
MerchantAvailability	ONLINE	0	Merchant available for matching
MerchantAvailability	OFFLINE	1	Merchant unavailable
ChannelStatus	PENDING	0	Awaiting review
ChannelStatus	APPROVED	1	Approved channel
ChannelStatus	REJECTED	2	Rejected; duplicate key freed
ChannelStatus	TERMINATED	3	Migrated/closed
ChannelAvailability	ACTIVE	0	Channel enabled
ChannelAvailability	INACTIVE	1	Channel disabled
FacetCutAction	Add	0	Register previously absent selectors
FacetCutAction	Replace	1	Move existing selectors to a new facet
FacetCutAction	Remove	2	Delete selector routes; facet address must be zero

Compatibility note

MerchantAccountStatus has no DORMANT member. Tests that use DORMANT=4 do not match the checked-in source.

Live-only order and dispute enums

The active ABI encodes these as uint8. Numeric names are reconstructed from the active mapping helpers and GraphQL schema because their Solidity declarations are absent from the contract repository.

Enum	Member	Value	Meaning
OrderType	BUY	0	Buy USDC order; mapping/schema interpretation
OrderType	SELL	1	Sell USDC order; mapping/schema interpretation
OrderStatus	CREATED	0	Created and awaiting acceptance
OrderStatus	ACCEPTED	1	Merchant/channel accepted
OrderStatus	PAID	2	Payment marked/confirmed
OrderStatus	COMPLETED	3	Settlement completed
OrderStatus	CANCELLED	4	Cancelled
DisputeStatus	NONE	0	No active dispute
DisputeStatus	OPEN	1	Dispute raised
DisputeStatus	SETTLED	2	Dispute/risk settled
DisputeResult	NONE	0	No result
DisputeResult	USER_WINS	1	Resolved for user
DisputeResult	MERCHANT_WINS	2	Resolved for merchant

Struct catalog

Struct	Fields in declaration order
Merchant	wallet:address; accountStatus:enum; availability:enum; usdcLiquidity:uint256; unstakePending:bool; unstakeRequestedAmount:uint256; telegramUsername:string; registeredAt:uint256; channelIds:bytes32[]
PaymentChannel	channelId:bytes32; merchant:address; bankName:string; accountLast4:string; upId:string; label:string; status:enum; availability:enum; fiatBalance:uint256; appliedAt:uint256; reviewedAt:uint256
PlatformConfig	admin:address; usdcToken:address; paused:bool; minMerchantStakeUsdc:uint256; initialized:bool
AppStorage	config; merchants mapping; merchantList; channels mapping; channelDuplicateGuard mapping; _reentrancyStatus
FacetCut	facetAddress:address; action:FacetCutAction; functionSelectors:bytes4[]
Facet	facetAddress:address; functionSelectors:bytes4[]
FacetAddressAndPosition	facetAddress:address; functionSelectorPosition:uint96
FacetFunctionSelectors	functionSelectors:bytes4[]; facetAddressPosition:uint256
DiamondStorage	selectorToFacetAndPosition mapping; facetFunctionSelectors mapping; facetAddresses array; supportedInterfaces mapping; contractOwner

Active ABI tuple evolution

Tuple	Active ABI fields	Difference from source
PlatformConfig	admin; usdcToken; paused; minMerchantStakeUsdc; initialized	Same five fields as checked-in source
Merchant	Source 9 fields + reservedUsdc + riskUsdc	Two active-ABI trailing additions
PaymentChannel	Source 11 fields + __deprecated_dailyLimitUsdc; __deprecated_monthlyLimitUsdc; dailyVolumeUsed; dailyWindowStart; monthlyVolumeUsed; monthlyWindowStart; reservedFiat	Seven active-ABI trailing additions
Order	orderId; orderType; status; user; merchant; channelId; usdcAmount; fiatAmount; price; createdAt; acceptedAt; paidAt; completedAt; cancelledAt; disputeExpiresAt; disputeStatus; disputeResolver; disputeResult; assignedMerchants; riskReleased; orderNumber	21-field live ABI tuple; Solidity declaration absent from source repo

Storage boundary

Trailing ABI fields suggest append-only evolution, but ABI tuples cannot reveal top-level AppStorage mapping seeds. Exact live storage remains unknown.

Exact AppStorage layout

All business facets inherit Modifiers, whose first state variable is AppStorage internal s. Therefore the struct begins at Diamond slot zero.

Slot	Field	Encoding / note
0	config.admin	address at byte offset 0
1	config.usdcToken + config.paused	address offset 0; bool offset 20
2	config.minMerchantStakeUsdc	uint256
3	config.initialized	bool
4	merchants	mapping(address => Merchant) seed
5	merchantList	dynamic array length; data at keccak256(5)
6	channels	mapping(bytes32 => PaymentChannel) seed
7	channelDuplicateGuard	mapping(bytes32 => bool) seed
8	_reentrancyStatus	uint256: 1 not-entered; 2 entered

Historic collision risk

An older layout placed mappings at slots 3/4/5/6. Adding config.initialized moved them to 4/5/6/7. This is an upgrade-breaking shift, not an append.

Nested mapping and dynamic layouts

Merchant mapping

```
q = keccak256(abi.encode(wallet, uint256(4)))
q+0 wallet + accountStatus + availability (packed)
q+1 usdcLiquidity
q+2 unstakePending
q+3 unstakeRequestedAmount
q+4 telegramUsername string descriptor/data
q+5 registeredAt
q+6 channelIds length; elements at keccak256(q+6)
```

PaymentChannel mapping

```
q = keccak256(abi.encode(channelId, uint256(6)))
q+0 channelId; q+1 merchant
q+2..q+5 bankName, accountLast4, upiId, label
q+6 status + availability (packed)
q+7 fiatBalance; q+8 appliedAt; q+9 reviewedAt
duplicateKey = keccak256(wallet || normalizeBankName(bankName) || accountLast4)
guard value at keccak256(abi.encode(duplicateKey, uint256(7)))
```


Diamond routing storage

```
P = keccak256('diamond.standard.diamond.storage')
P = 0xc8fcad8db84d3cc18b4c41d551ea0ee66dd599cde068d998e57d5e09332c131c
P+0 selectorToFacetAndPosition mapping seed
P+1 facetFunctionSelectors mapping seed
P+2 facetAddresses dynamic array
P+3 supportedInterfaces mapping seed
P+4 contractOwner
```

Selector arrays and facet arrays use swap-and-pop during replace/remove. This region is distinct from the slot-zero business AppStorage.

State variables and constants

Symbol / region	Type / value	Role
Modifiers.s	AppStorage internal	Slot-zero shared business storage used by ConfigFacet, MerchantFacet, and initializer.
DIAMOND_STORAGE_POSITION	0xc8fcad8db84d3cc18b4c41d551ea0ee66dd599cde068d998e57d5e09332c131c	Hashed EIP-2535 routing and owner region.
_NOT_ENTERED	1	Expected idle value for shared reentrancy status.
_ENTERED	2	Locked value during protected external-token paths.
merchantList	address[]	Append-only registration list; source exposes the complete array.
channelDuplicateGuard	mapping(bytes32=>bool)	Prevents duplicate wallet + normalized bank + accountLast4 channels.
PlatformConfig.initialized	bool	Guards DiamondInit; its insertion is the historical layout-shift risk.
Diamond receive balance	native ETH	Proxy can accept ETH but source has no withdrawal/rescue function.
USDC token balance	IERC20 at config.usdcToken	Diamond custody; source accounting uses merchant.usdcLiquidity.

Source function catalog - 36 routed selectors

Access and behavior are derived from the checked-in Solidity. All integrations call these selectors at the Diamond address.

Facet	Selector	Signature	Mutability	Access / guards	State or external effect
DiamondCutFacet	0x1f931c1c	diamondCut((address,uint8,bytes4[]),address,bytes)	nonpayable	Diamond owner	Add/replace/remove selector routes; optional initializer delegatecall.
DiamondLoupeFacet	0xcdffacc6	facetAddress(bytes4)	view	Permissionless view	Resolve selector to implementation address.
DiamondLoupeFacet	0x52ef6b2c	facetAddresses()	view	Permissionless view	Return all registered facet addresses.
DiamondLoupeFacet	0xadfca15e	facetFunctionSelectors(address)	view	Permissionless view	Return selectors routed to one facet.
DiamondLoupeFacet	0x7a0ed627	facets()	view	Permissionless view	Return every facet address and selector array.
DiamondLoupeFacet	0x01ffc9a7	supportsInterface(bytes4)	view	Permissionless view	Read ERC-165 support mapping.
OwnershipFacet	0x8da5cb5b	owner()	view	Permissionless view	Read Diamond contract owner.
OwnershipFacet	0xf2fde38b	transferOwnership(address)	nonpayable	Diamond owner	Write Diamond owner; zero address is permitted.
ConfigFacet	0xc3f909d4	getConfig()	view	Permissionless view	Return PlatformConfig.
ConfigFacet	0x6b78c29b	pausePlatform()	nonpayable	Platform admin	Set paused=true and emit PlatformPaused.
ConfigFacet	0x64ec2ceb	setMinMerchantStake(uint256)	nonpayable	Platform admin	Replace minimum raw USDC stake.
ConfigFacet	0x3397d9a2	transferPlatformAdmin(address)	nonpayable	Platform admin	Replace business admin; rejects zero address.
ConfigFacet	0x1a9ba7eb	unpausePlatform()	nonpayable	Platform admin	Set paused=false and emit PlatformUnpaused.
MerchantFacet	0x1fee2a96	addPaymentChannel(string,string,string,string)	nonpayable	Active merchant; notPaused	Normalize/guard bank key; create PENDING/INACTIVE channel.
MerchantFacet	0x3d58ff4a	approveChannel(bytes32)	nonpayable	Platform admin	PENDING -> APPROVED/ACTIVE; set reviewedAt.
MerchantFacet	0xbb634d55	approveMerchantUnstake(address)	nonpayable	Platform admin; nonReentrant	Clear request; transfer USDC out; restore ACTIVE/OFFLINE.
MerchantFacet	0x30321dcc	blacklistMerchant(address)	nonpayable	Platform admin	Cancel request; set BLACKLISTED/OFFLINE.
MerchantFacet	0xd91b0a8d	clearMerchantDispute(address)	nonpayable	Platform admin	DISPUTED -> ACTIVE; stays OFFLINE.
MerchantFacet	0xcb82cc8f	depositStake(uint256)	nonpayable	Merchant; notPaused; nonReentrant	Transfer USDC in and increase tracked liquidity.
MerchantFacet	0x8ce2df51	getAllMerchants()	view	Permissionless view	Return merchantList addresses.
MerchantFacet	0x831c2b82	getChannel(bytes32)	view	Permissionless view	Return channel by ID.
MerchantFacet	0xb2734eaf	getMerchant(address)	view	Permissionless view	Return Merchant by wallet.
MerchantFacet	0xb4de411c	getMerchantChannels(address)	view	Permissionless view	Materialize all channels for wallet.
MerchantFacet	0xae180328	getMyChannels()	view	Permissionless view	Self-call getMerchantChannels(msg.sender).
MerchantFacet	0x21527e50	getMyProfile()	view	Permissionless view	Return caller Merchant.
MerchantFacet	0x8307d08b	getPendingChannels()	view	Permissionless view	Two-pass scan of all merchants/channels; O(total channels).
MerchantFacet	0xa6485ccd	goOffline()	nonpayable	Registered merchant	Set OFFLINE.

Facet	Selector	Signature	Mutability	Access / guards	State or external effect
MerchantFacet	0x6e5b676b	goOnline()	nonpayable	Active merchant; notPaused	Require ACTIVE and current minimum; set ONLINE.
MerchantFacet	0x0586296c	migrateAndTerminate(bytes32,bytes32)	nonpayable	Owner merchant; notPaused	Move fiatBalance; terminate source; free duplicate key.
MerchantFacet	0xb00c52b0	registerMerchant(uint256,string)	nonpayable	Public; notPaused; nonReentrant	Transfer stake in; initialize Merchant; append merchantList.
MerchantFacet	0x38a9f5df	rejectChannel(bytes32)	nonpayable	Platform admin	PENDING -> REJECTED/INACTIVE; free duplicate key.
MerchantFacet	0x66d3b61c	rejectMerchantUnstake(address)	nonpayable	Platform admin	Clear request; restore ACTIVE; stays OFFLINE.
MerchantFacet	0x8e0540de	setMerchantDisputed(address)	nonpayable	Platform admin	ACTIVE -> DISPUTED/OFFLINE.
MerchantFacet	0xb7889c93	setPaymentChannelActive(bytes32)	nonpayable	Owner merchant; notPaused	Approved caller-owned channel -> ACTIVE.
MerchantFacet	0x1dcad144	setPaymentChannelInactive(bytes32)	nonpayable	Owner merchant	Approved caller-owned channel -> INACTIVE.
MerchantFacet	0xbcd9d861	withdrawStake()	nonpayable	Registered merchant	Request full unstake; set INACTIVE/OFFLINE; no token transfer.

Internal functions and initializer

Library / contract	Functions	Purpose
LibMerchants	generateChannelId; normalizeBankName; isAllAsciiDigits; _isAsciiSpace	Pure ID/normalization/validation helpers; internal/private and inlined.
LibDiamond	diamondStorage; setContractOwner; contractOwner; enforcesContractOwner	Resolve hashed storage and ownership.
LibDiamond	diamondCut; addFunctions; replaceFunctions; removeFunctions; addFacet; addFunction; removeFunction	Selector/facet array and mapping mutation.
LibDiamond	initializeDiamondCut; enforceHasContractCode	Optional delegatecall initializer and code checks.
DiamondInit	init(address,uint256)	Set interfaces, USDC, minimum, paused=false, admin=msg.sender, initialized=true, lock=1.
Diamond	constructor; fallback; receive	Bootstrap cut route, runtime proxy, plain ETH receiver.

Live facet address and selector counts

Facet	Live address	Selectors	Function names
DiamondCutFacet	0x13E3B3C63362B1cad5430c3745dC96130E7a5117	1	diamondCut
DiamondLoupeFacet	0x3D50E8DF96e7F43a8570A9e54C42F8b559fffB58	5	facetAddress, facetAddresses, facetFunctionSelectors, facets, supportsInterface
OwnershipFacet	0x2c63a6234D1a587D7b160FF96f703c1097f7b30	2	owner, transferOwnership
ConfigFacet	0xcF9510e42511014FaB632238Dbf5250562C61D83	15	addEligibleMerchant, clearEligibleMerchants, getChannelLimitDefaults, getConfig, getEligibleMerchants, getOrderPricing, isEligibleMerchant, pausePlatform, removeEligibleMerchant, setDefaultChannelLimits, setDisputeWindow, setMinMerchantStake, setOrderPricing, transferPlatformAdmin, unpausePlatform
MerchantFacet	0x2C1e028064c18aD316Fa8Fa69d1B328cC219E97D	24	addPaymentChannel, approveChannel, approveMerchantUnstake, blacklistMerchant, clearMerchantDispute, depositStake, getAllMerchants, getChannel, getChannelLimits, getMerchant, getMerchantChannels, getMyChannels, getMyProfile, getPendingChannels, goOffline, goOnline, migrateAndTerminate, registerMerchant, rejectChannel, rejectMerchantUnstake, setMerchantDisputed, setPaymentChannelActive, setPaymentChannelInactive, withdrawStake
OrderFacet	0xCCA73B72b83FDccfBF4294224c3ccc305df4Fb	16	acceptOrder, cancelOrder, confirmPayment, createBuyOrder, createSellOrder, getAssignedMerchants, getChannelFiat, getMerchantBalances, getMerchantOrders, getOrder, getOrderIds, getUserOrders, markPaymentSent, raiseDispute, resolveDispute, settleOrder

Loupe evidence

Facet addresses/selectors were resolved read-only from the active Base Sepolia Diamond. `init(...)` is present in the ABI but absent from loupe routing.

Live routed function catalog - 63 selectors

Facet	Selector	Signature	Mutability	Outputs
DiamondCutFacet	0x1f931c1c	diamondCut((address,uint8,bytes4[]),address,bytes)	nonpayable	-
DiamondLoupeFacet	0xcdffacc6	facetAddress(bytes4)	view	facetAddress_:address
DiamondLoupeFacet	0x52ef6b2c	facetAddresses()	view	facetAddresses_:address[]
DiamondLoupeFacet	0xadfca15e	facetFunctionSelectors(address)	view	facetFunctionSelectors_:bytes4[]
DiamondLoupeFacet	0x7a0ed627	facets()	view	facets_:(address,bytes4[])
DiamondLoupeFacet	0x01ffc9a7	supportsInterface(bytes4)	view	bool
OwnershipFacet	0x8da5cb5b	owner()	view	owner_:address
OwnershipFacet	0xf2fde38b	transferOwnership(address)	nonpayable	-
ConfigFacet	0x09ec0f24	addEligibleMerchant(address)	nonpayable	-
ConfigFacet	0x3551ac6c	clearEligibleMerchants()	nonpayable	-
ConfigFacet	0xbf284d84	getChannelLimitDefaults()	view	dailyUsdc:uint256, monthlyUsdc:uint256
ConfigFacet	0xc3f909d4	getConfig()	view	(address,address,bool,uint256,bool)
ConfigFacet	0x2f583d4b	getEligibleMerchants()	view	address[]
ConfigFacet	0xab211bd9	getOrderPricing()	view	buyPriceInrPerUsdc:uint256, sellPriceInrPerUsdc:uint256, disputeWindowSeconds:uint256
ConfigFacet	0x903eadc0	isEligibleMerchant(address)	view	bool
ConfigFacet	0x6b78c29b	pausePlatform()	nonpayable	-
ConfigFacet	0x6a96f84d	removeEligibleMerchant(address)	nonpayable	-
ConfigFacet	0x892b8a9c	setDefaultChannelLimits(uint256,uint256)	nonpayable	-
ConfigFacet	0x332226d0	setDisputeWindow(uint256)	nonpayable	-
ConfigFacet	0x64ec2ceb	setMinMerchantStake(uint256)	nonpayable	-
ConfigFacet	0xf7260e6e	setOrderPricing(uint256,uint256)	nonpayable	-
ConfigFacet	0x3397d9a2	transferPlatformAdmin(address)	nonpayable	-
ConfigFacet	0x1a9ba7eb	unpausePlatform()	nonpayable	-
MerchantFacet	0x1fee2a96	addPaymentChannel(string,string,string,string)	nonpayable	-
MerchantFacet	0x3d58ff4a	approveChannel(bytes32)	nonpayable	-
MerchantFacet	0xbb634d55	approveMerchantUnstake(address)	nonpayable	-
MerchantFacet	0x30321dcc	blacklistMerchant(address)	nonpayable	-
MerchantFacet	0xd91b0a8d	clearMerchantDispute(address)	nonpayable	-

Facet	Selector	Signature	Mutability	Outputs
MerchantFacet	0xcb82cc8f	depositStake(uint256)	nonpayable	-
MerchantFacet	0x8ce2df51	getAllMerchants()	view	address[]
MerchantFacet	0x831c2b82	getChannel(bytes32)	view	(bytes32,address,string,string,string,string,uint8,uint8,uint256,uint256,uint256,uint256,uint256,uint256,uint256,uint256)
MerchantFacet	0x5b020623	getChannelLimits(bytes32)	view	dailyLimitUsdc:uint256, dailyVolumeUsed:uint256, dailyResetsAt:uint256, monthlyLimitUsdc:uint256, monthlyVolumeUsed:uint256, monthlyResetsAt:uint256
MerchantFacet	0xb2734eaf	getMerchant(address)	view	(address,uint8,uint8,uint256,bool,uint256,string,uint256,bytes32[],uint256,uint256)
MerchantFacet	0xb4de411c	getMerchantChannels(address)	view	(bytes32,address,string,string,string,string,uint8,uint8,uint256,uint256,uint256,uint256,uint256,uint256,uint256,uint256)
MerchantFacet	0xae180328	getMyChannels()	view	(bytes32,address,string,string,string,string,uint8,uint8,uint256,uint256,uint256,uint256,uint256,uint256,uint256,uint256)
MerchantFacet	0x21527e50	getMyProfile()	view	(address,uint8,uint8,uint256,bool,uint256,string,uint256,bytes32[],uint256,uint256)
MerchantFacet	0x8307d08b	getPendingChannels()	view	bytes32[]
MerchantFacet	0xa6485ccd	goOffline()	nonpayable	-
MerchantFacet	0x6e5b676b	goOnline()	nonpayable	-
MerchantFacet	0x0586296c	migrateAndTerminate(bytes32,bytes32)	nonpayable	-
MerchantFacet	0xb00c52b0	registerMerchant(uint256,string)	nonpayable	-
MerchantFacet	0x38a9f5df	rejectChannel(bytes32)	nonpayable	-
MerchantFacet	0x66d3b61c	rejectMerchantUnstake(address)	nonpayable	-
MerchantFacet	0x8e0540de	setMerchantDisputed(address)	nonpayable	-
MerchantFacet	0xb7889c93	setPaymentChannelActive(bytes32)	nonpayable	-
MerchantFacet	0x1dcad144	setPaymentChannelInactive(bytes32)	nonpayable	-
MerchantFacet	0xbcd9d861	withdrawStake()	nonpayable	-
OrderFacet	0xd6039a61	acceptOrder(bytes32,bytes32)	nonpayable	-
OrderFacet	0x7489ec23	cancelOrder(bytes32)	nonpayable	-
OrderFacet	0x3611d088	confirmPayment(bytes32)	nonpayable	-
OrderFacet	0x84ce1bfc	createBuyOrder(uint256)	nonpayable	orderId:bytes32, assigned:address[]
OrderFacet	0x3c81c4b8	createSellOrder(uint256)	nonpayable	orderId:bytes32, assigned:address[]
OrderFacet	0x7372f2f1	getAssignedMerchants(bytes32)	view	address[]
OrderFacet	0x1e3e148d	getChannelFiat(bytes32)	view	totalFiat:uint256, reservedFiat:uint256, unreservedFiat:uint256

Facet	Selector	Signature	Mutability	Outputs
OrderFacet	0xeb0817c5	getMerchantBalances(address)	view	totalUsdc:uint256, reservedUsdc:uint256, riskUsdc:uint256, unreservedUsdc:uint256
OrderFacet	0x4ebac543	getMerchantOrders(address)	view	bytes32[]
OrderFacet	0x5778472a	getOrder(bytes32)	view	(bytes32,uint8,uint8,address,address,bytes32,uint256,uint256,uint256,uint256,uint256,uint256,uint256,uint256,uint8,address,uint8,address[],bool,uint256)
OrderFacet	0x9e0acf8f	getOrderIds()	view	bytes32[]
OrderFacet	0x63c69f08	getUserOrders(address)	view	bytes32[]
OrderFacet	0x3af1b286	markPaymentSent(bytes32)	nonpayable	-
OrderFacet	0xe14f5b7d	raiseDispute(bytes32)	nonpayable	-
OrderFacet	0xb641237c	resolveDispute(bytes32,uint8)	nonpayable	-
OrderFacet	0x49085d8c	settleOrder(bytes32)	nonpayable	-

ABI errors and source revert surface

Custom error	Signature
InitializationFunctionReverted	InitializationFunctionReverted(address,bytes)
SafeERC20FailedOperation	SafeERC20FailedOperation(address)

Domain reverts are Error(string); grouped without changing wording categories

```
Diamond: Function does not exist
LibDiamond: Must be contract owner
LibDiamondCut: No selectors in facet to cut / incorrect action / target has no code
LibDiamondCut: Can't add existing / replace same / remove missing or immutable function
Already initialized; Zero USDC; Not admin; Platform is paused; ReentrancyGuard: reentrant call
Merchant: Already registered; Below minimum stake; Telegram required; Not a merchant; Account not active
Merchant: Unstake pending; Amount must be > 0; Not active; Already pending; No liquidity; Below min USDC liquidity
Channel: required fields; last4 length/digits; duplicate; ownership; status; pending; not found; same channel
Admin: No unstake request; Not awaiting unstake; Liquidity mismatch; Already blacklisted; Must be ACTIVE; Not under dispute
```

Active ABI events - 36

Indexed arguments form log topics; other arguments are ABI-encoded in event data. The active subgraph listens to a subset according to subgraph.yaml.

Domain	Event	Indexed fields	Data fields
Config	DefaultChannelLimitsUpdated	-	dailyUsdc:uint256, monthlyUsdc:uint256
Config	DisputeWindowUpdated	-	disputeWindowSeconds:uint256
Config	EligibleMerchantAdded	merchant:address	-
Config	EligibleMerchantRemoved	merchant:address	-
Config	EligibleMerchantsCleared	-	-
Config	MinMerchantStakeUpdated	-	newMinStakeUsdc:uint256
Config	OrderPricingUpdated	-	buyPriceInrPerUsdc:uint256, sellPriceInrPerUsdc:uint256
Config	PlatformAdminTransferred	previousAdmin:address, newAdmin:address	-
Config	PlatformPaused	by:address	-
Config	PlatformUnpaused	by:address	-
Diamond core	DiamondCut	-	_diamondCut:(address,uint8,bytes4[]), _init:address, _calldata:bytes
Diamond core	OwnershipTransferred	previousOwner:address, newOwner:address	-
Merchant	AvailabilityChanged	wallet:address	availability:uint8
Merchant	ChannelAdded	channelId:bytes32, wallet:address	-
Merchant	ChannelApproved	channelId:bytes32, wallet:address	-
Merchant	ChannelAvailabilityChanged	channelId:bytes32, wallet:address	availability:uint8
Merchant	ChannelRejected	channelId:bytes32, wallet:address	-
Merchant	ChannelTerminated	channelId:bytes32, wallet:address	-
Merchant	FiatMigrated	fromChannelId:bytes32, toChannelId:bytes32, wallet:address	amount:uint256
Merchant	MerchantBlacklisted	wallet:address	-
Merchant	MerchantDisputeCleared	wallet:address	-
Merchant	MerchantDisputed	wallet:address	-
Merchant	MerchantRegistered	wallet:address	usdcLiquidity:uint256
Merchant	UnstakeRequestRejected	wallet:address	-
Merchant	UnstakeRequested	wallet:address	amount:uint256
Merchant	UsdcDeposited	wallet:address	amount:uint256
Merchant	UsdcWithdrawn	wallet:address	amount:uint256

Domain	Event	Indexed fields	Data fields
Order	DisputeRaised	orderId:bytes32, by:address	raisedAt:uint256
Order	DisputeResolved	orderId:bytes32, resolver:address	result:uint8, resolvedAt:uint256
Order	OrderAccepted	orderId:bytes32, merchant:address, channelId:bytes32	acceptedAt:uint256
Order	OrderAssigned	orderId:bytes32, merchant:address	assignedAt:uint256
Order	OrderCancelled	orderId:bytes32, by:address	cancelledAt:uint256
Order	OrderCompleted	orderId:bytes32, merchant:address	completedAt:uint256, disputeExpiresAt:uint256
Order	OrderCreated	orderId:bytes32, user:address	orderType:uint8, usdcAmount:uint256, fiatAmount:uint256, price:uint256, createdAt:uint256, orderNumber:uint256
Order	OrderPaid	orderId:bytes32, by:address	paidAt:uint256
Order	OrderRiskReleased	orderId:bytes32, merchant:address	usdcAmount:uint256

Roles, pausing, and trust boundaries

Boundary	Capability	Important limitation
Diamond owner	diamondCut; transferOwnership	Ultimate authority: can replace selectors and run arbitrary initializer delegatecall.
Platform admin	Pause/config and merchant administrative actions	Separate slot/role, but Diamond owner can indirectly replace its enforcement.
Merchant	Stake, availability, own channels, withdrawal request	Blacklisting has no reversal; approved full unstake restores ACTIVE with zero liquidity.
User / public	Views and public order entrypoints where deployed	Live OrderFacet source/storage is not present in this repository.
Pause flag	Blocks only functions carrying notPaused	withdrawStake, goOffline, deactivation, admin actions, views, upgrades remain available.
Reentrancy lock	Protects register, deposit, approve unstake	Shared status must be initialized to 1.
USDC custody	SafeERC20 transfer in/out	No balance-delta validation; token address/decimals not verified; no rescue.

Direct facet calls

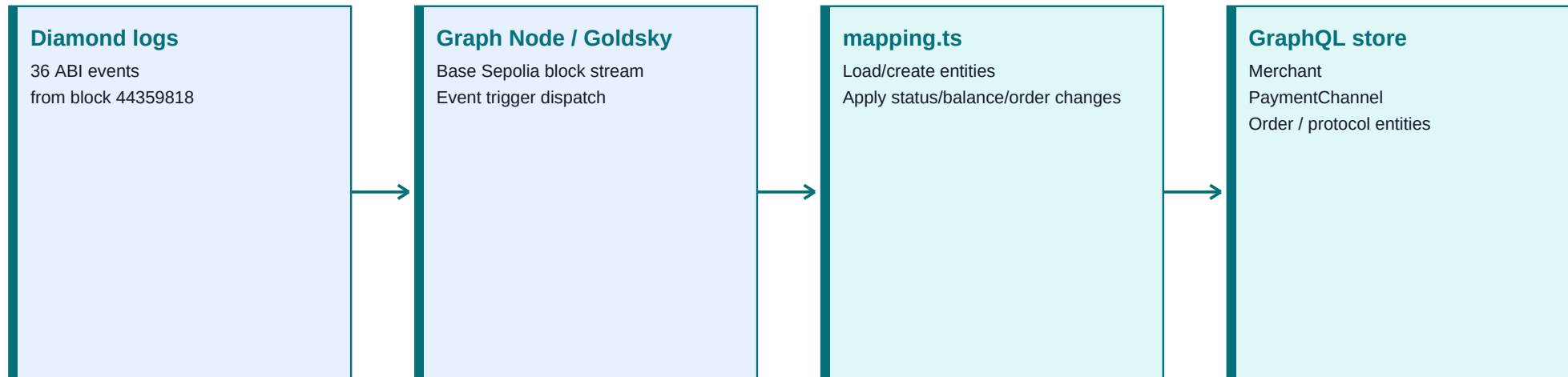
Calling a facet address bypasses the Diamond's state and observes uninitialized facet-local storage. Always target the Diamond.

Errors, scale, and operational hazards

- Fallback rejects unknown selectors with 'Diamond: Function does not exist'; facet revert data is forwarded exactly.
- LibDiamond enforces owner, nonempty cuts, correct Add/Replace/Remove invariants, target code, and immutable-function protection.
- Business failures use Error(string); SafeERC20 can surface SafeERC20FailedOperation(address).
- getPendingChannels performs two full scans over all merchants/channels and can become unusable as state grows.
- Plain ETH is accepted by receive(), and direct ERC-20 transfers are possible, but no rescue function exists.
- withdrawStake does not inspect fiat obligations and does not deactivate payment channels.
- The current source has no function that originates channel fiatBalance; it only migrates existing value.
- transferOwnership(address(0)) is allowed and can permanently disable normal upgrades.

Subgraph indexing pipeline

The GraphQL API is eventually consistent and is not a transaction authority.



Consistency boundary

A successful transaction receipt precedes subgraph visibility. UIs should either wait/poll for indexing or optimistically update and reconcile; they must not assume an immediate GraphQL refresh contains the new event.

Active indexing configuration

Property	Active value	Implication
Network	base-sepolia	Chain ID 84532 deployment generation
Diamond	0xF40AD901CCfB5E5EdC5162D6Ac7DDd5Ed5899F3A	Single event source address
Start block	44359818	Earlier logs are intentionally excluded
ABI	p2pflow-subgraph/abis/Diamond.json	64 function entries; 36 event entries
Schema	Materialized entities	Merchant/channel/order state, not raw event-table query names
Legacy manifest	Ethereum Sepolia 0x456850ff3Eb1bA5c3312fA97A47307992103855E	Historical deployment; do not mix endpoints or IDs

Admin incompatibility

Admin useSubgraph queries raw event tables while the active schema is materialized. Those GraphQL operations fail against the active endpoint.

GraphQL schema enums - 12

Enum	Members in declaration order
MerchantAccountStatus	ACTIVE, INACTIVE, BLACKLISTED, DISPUTED
MerchantAvailability	ONLINE, OFFLINE
ChannelStatus	PENDING, APPROVED, REJECTED, TERMINATED
ChannelAvailability	ACTIVE, INACTIVE
MerchantActivityKind	REGISTERED, USDC_DEPOSITED, UNSTAKE_REQUESTED, UNSTAKE_REJECTED, USDC_WITHDRAWN, AVAILABILITY_ONLINE, AVAILABILITY_OFFLINE
ChannelEventKind	ADDED, APPROVED, REJECTED, AVAILABILITY_CHANGED, TERMINATED
PlatformEventKind	PAUSED, UNPAUSED, MIN_STAKE_UPDATED, ADMIN_TRANSFERRED, OWNERSHIP_TRANSFERRED, DIAMOND_CUT, DEFAULT_CHANNEL_LIMITS_UPDATED, ORDER_PRICING_UPDATED, DISPUTE_WINDOW_UPDATED, ELIGIBLE_MERCHANT_ADDED, ELIGIBLE_MERCHANT_REMOVED, ELIGIBLE_MERCHANTS_CLEARED
OrderType	BUY, SELL
OrderStatus	CREATED, ACCEPTED, PAID, COMPLETED, CANCELLED
DisputeStatus	NONE, OPEN, SETTLED
DisputeResult	NONE, USER_WINS, MERCHANT_WINS
OrderEventKind	CREATED, ASSIGNED, ACCEPTED, PAID, COMPLETED, CANCELLED, RISK_RELEASED, DISPUTE_RAISED, DISPUTE_RESOLVED

Projection note

Order/dispute numeric meanings are mapping assumptions mirrored by the schema; their Solidity enum declarations are absent from the checked-in contract source.

GraphQL entity catalog - 12 entities

Entity	Lifecycle	Primary fields / purpose
Platform	Mutable singleton	id; admin; usdcToken; paused; minMerchantStakeUsdc; defaultChannelDailyLimitUsdc; defaultChannelMonthlyLimitUsdc; buyPriceInrPerUsdc; sellPriceInrPerUsdc; disputeWindowSeconds; eligibleMerchants; 12 merchant/channel counters; totalUsdcCustodied; lastUpdatedBlock; lastUpdatedTimestamp; events
Merchant	Mutable snapshot	id; wallet; telegramUsername; accountStatus; availability; usdcLiquidity; unstakePending; unstakeRequestedAmount; totalDeposited; totalWithdrawn; channelCount; activeChannelCount; reservedUsdc; riskUsdc; unreservedUsdc; registration provenance; blacklistedAt; disputedAt; freshness; derived relations
PaymentChannel	Mutable snapshot	id; channelId; merchant; bankName; accountLast4; upId; label; status; availability; fiatBalance; reservedFiat; dailyVolumeUsed; dailyWindowStart; monthlyVolumeUsed; monthlyWindowStart; lifecycle/freshness; derived events/migrations
Order	Mutable snapshot	id; orderId; orderNumber; orderType; status; user; merchant; channel; usdcAmount; fiatAmount; price; lifecycle timestamps; disputeStatus/result/resolver/timestamps; riskReleased; provenance/freshness; assignments/events/dispute
OrderAssignment	Mutable relation	id=orderId merchant; order; merchant; merchantAddress; assignedAt; accepted; block; timestamp; txHash
Dispute	Mutable one-to-one	id=orderId; order; raisedBy; raisedAt; status; resolver; result; resolvedAt; block; timestamp; txHash
MerchantActivity	Immutable history	id=txHash logIndex; merchant; kind; optional amount; block; timestamp; txHash; logIndex
MerchantStatusChange	Immutable history	id; merchant; previousStatus; newStatus; block; timestamp; txHash; logIndex
ChannelEvent	Immutable history	id; channel; merchant; kind; optional availability; block; timestamp; txHash; logIndex
FiatMigration	Immutable history	id; merchant; fromChannel; toChannel; amount; block; timestamp; txHash; logIndex
PlatformEvent	Immutable history	id; platform; kind; optional actor/admin/owner/minimum/limits/pricing/window/eligibleMerchant fields; block; timestamp; txHash; logIndex
OrderEvent	Immutable history	id; order; kind; optional actor/merchant/usdcAmount/fiatAmount/disputeExpiresAt/disputeResult; block; timestamp; txHash; logIndex

35 handlers and server-side enrichment

Counts below are approximate Diamond view calls made by Goldsky while indexing one event. They are server-side RPC traffic, not browser Thirdweb calls. DiamondCut is the one ABI event with no handler.

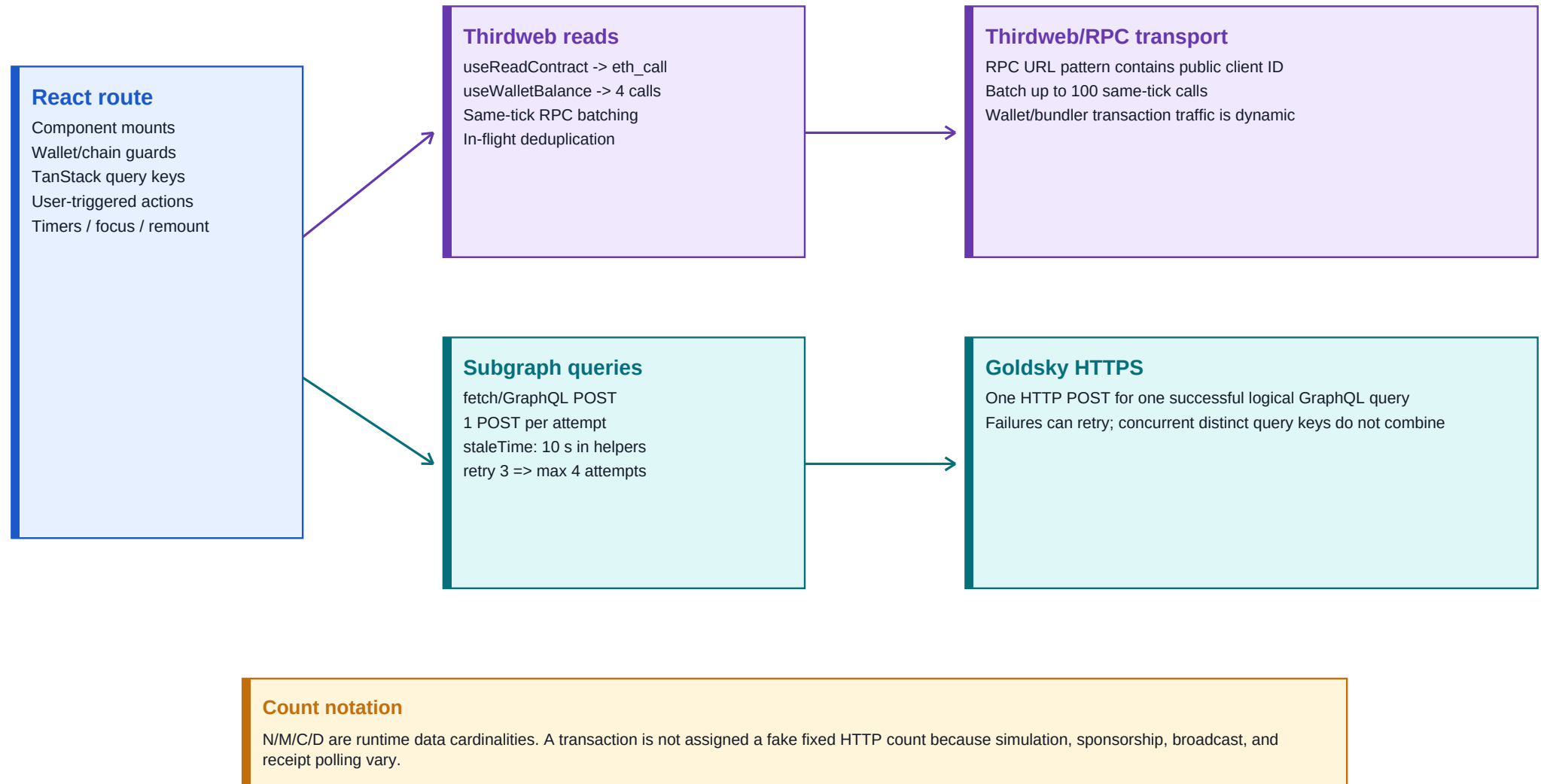
Event group	Entity mutations	Indexer view calls / event
MerchantRegistered	Merchant; Platform counters/custody; MerchantActivity	6
USDC/unstake/status merchant events (8)	Merchant totals/status; Platform buckets/custody; activity/history	4 or 6 each
ChannelAdded	PaymentChannel; merchant/platform counts; ChannelEvent	7
ChannelApproved / Rejected / Terminated	Channel refresh; status/active counters; ChannelEvent	7 each
ChannelAvailabilityChanged	Availability and active counters; ChannelEvent	0 or 4
FiatMigrated	Refresh source/destination; immutable FiatMigration	6
Platform config/ownership/eligibility events (11)	Platform values/list; immutable PlatformEvent	4 each
OrderCreated / Paid / Cancelled / DisputeRaised	Order/Dispute snapshots and OrderEvent	0
OrderAssigned	Assignment + defensive Merchant; OrderEvent	0 or 2
OrderAccepted / Completed	Order + assignment; merchant/channel refresh; OrderEvent	up to 5
OrderRiskReleased / DisputeResolved	Risk/dispute state; merchant refresh; OrderEvent	up to 2

Mixed consistency model

Counters are event-derived, while configuration and projections are opportunistically refreshed from live views. Missing prerequisite entities can cause handlers to return early.

UI request lifecycle and counting model

The report counts logical operations, HTTP attempts, and recurring timers separately.



Provider and query-client lifecycle

Client	Effective query behavior	Write/read consequence
User	Inner app QueryClient: staleTime 0, retry 3, focus/mount/reconnect refetch	Thirdweb provider receipt wait/read invalidation is bypassed; writes generally resolve after broadcast.
Merchant	Same inner default QueryClient; subgraph hooks override staleTime 10 s and focus=false	Register/add/channel writes do not invalidate GraphQL/channel state; guard may remain stale.
Admin	Thirdweb client defaults: contract reads stale about 60 s; GraphQL helpers stale 10 s	Injected-wallet writes wait/poll receipts and invalidate active reads; chain/ABI mismatch still dominates.
All	React StrictMode enabled; pending same-key reads normally dedupe	Raw useEffect requests can replay; abort-aware GraphQL can be sent, aborted, then restarted in development.

Root lifecycle defect

In user and merchant apps, ThirdwebProvider is outside the app-created QueryClientProvider. Hooks therefore use the inner default client instead of Thirdweb's intended 60-second query client.

Endpoint and URL catalog

Patterns only: literal environment values and client identifiers are intentionally excluded.

Class	Configuration/provider	Redacted URL pattern	Use
EVM RPC	Thirdweb default	https://84532.rpc.thirdweb.com/<client-id>	User/merchant JSON-RPC reads, preflight, broadcast and receipt polling.
Admin EVM RPC	Thirdweb or VITE_ALCHEMY_RPC_URL	https://11155111.rpc.thirdweb.com/<client-id> or <override>	Current admin chain is Ethereum Sepolia although its Diamond/subgraph values are Base.
Embedded wallet	Thirdweb in-app wallet	https://embedded-wallet.thirdweb.com	User/merchant login and account lifecycle; traffic is strategy/session-dependent.
Bundler	Thirdweb EIP-7702	https://84532.bundler.thirdweb.com/v2	User/merchant sponsored flow; dynamic preflight/status polling. Admin does not activate it.
Chain metadata	Thirdweb	https://api.thirdweb.com/v1/chains/<chain-id>	Conditional GET; approximately five-minute SDK cache.
Social profile	Thirdweb	https://social.thirdweb.com/v1/profiles/<wallet>	AccountName and AccountAvatar can independently issue two GETs.
Analytics	Thirdweb	https://c.thirdweb.com/event	Conditional connection and transaction lifecycle events.
Embedded auth	Thirdweb	https://embedded-wallet.thirdweb.com/api/2024-05-05/login/<strategy>?clientid=<redacted>	Email/passkey/OAuth login and callback; strategy-dependent.
Linked accounts	Thirdweb	https://embedded-wallet.thirdweb.com/api/2024-05-05/accounts	Bearer GET for connected in-app wallet profiles.
ENS resolver RPC	Thirdweb	https://1.rpc.thirdweb.com/<client-id>	Conditional Ethereum-mainnet reverse lookup; shared/cacheable.
Insight	Thirdweb	https://insight.thirdweb.com	Only when wallet modal asset/history views are opened.
Bridge	Thirdweb	https://bridge.thirdweb.com/v1/tokens?...	Conditional insufficient-funds/pay flow.
Subgraph	Goldsky GraphQL	https://api.goldsky.com/api/public/<project>/subgraphs/<name>/<tag>/gn	One POST per logical attempt; concrete public project path redacted.
User REST	VITE_APP_BASE_URL / VITE_APP_BASE_LIVE_URL	<configured base>/...	FAQ/auth paths use split environment names; credentials and wallet headers enabled.
Merchant REST	VITE_APP_BASE_URL	<configured base>/...	Client exists; no active consumer found.
Admin REST	VITE_APP_BASE_URL / VITE_APP_BASE_LIVE_URL	<configured base>/...	FAQ path active; split config can fall through to frontend origin without a proxy.
QR image	goQR image service	https://api.qrserver.com/v1/create-qr-code/?...	One image GET per uncached mount; encodes UPI ID and amount to a third party.

Secret handling

Vite exposes VITE_* values to browser JavaScript. Never store Thirdweb secret keys, backend signing secrets, or private API credentials in them.

How request counts are measured

Operation	Logical count	HTTP / repetition note
One useReadContract	1 logical eth_call	Often coalesced with same-tick calls into one JSON-RPC batch POST, maximum 100.
User useWalletBalance	4 logical eth_call	balanceOf + name + symbol + decimals; normally one batch POST on a cold fetch.
Merchant USDC observer	1 logical eth_call	Direct balanceOf(address), not the four-read metadata helper.
TanStack query success	1 attempt	staleTime controls whether mount/focus/reconnect produces another execution.
TanStack query failure	Up to 4 attempts	Initial attempt plus retry:3 unless overridden; approximately 1/2/4-second backoff.
Subgraph helper query	1 POST per attempt	Cold success = 1 POST; persistent failure can reach 4 POSTs for each key.
Contract write	No fixed HTTP count	May include pay preflight, simulation, wallet/delegation, bundler, broadcast and receipt polling.
React StrictMode dev	May replay mounts/effects	Same-key pending queries usually dedupe; raw effects and abort/restart fetches may duplicate.
Connected ConnectButton	Conditional provider calls	Native balance, chain metadata, two social profile GETs, ENS and analytics depend on state/cache.

Count formulas

```
subgraph cold success:    logical queries x 1 HTTP POST
subgraph persistent failure: logical queries x (1 initial + 3 retries)
wallet balance cold fetch: 4 eth_call methods ~= 1 batched RPC POST
order screen over T seconds: 1 immediate + floor(T / 6) getOrder reads
transaction:              variable preflight + wallet/bundler + send + receipt polls
```

Admin UI route call matrix

Route	Cold logical calls	What calls	Caveat / repetition
/, /dashboard	4 base eth_call + D getDispute	getPlatformStats; getOpenDisputes; getPendingApplications; getAdmins; then one getDispute per ID	Legacy selectors are absent live; failures can retry up to 4 attempts per query.
/transactions/all	1 + on-demand	getPlatformStats mount; one getOrder for each newly submitted ID	No active polling.
/merchants	2 base + N + U or M	Both lists mount; applications add N getChannelApplication + U getMerchant; active adds M getMerchant	Hidden-tab list still runs; legacy types/signatures are incompatible.
/payments-channels	Intended 1 + M GraphQL POST; then C eth_call	AllMerchants; one channels query/merchant; one getChannel/channel	First raw-event query fails active-schema validation up to 4 attempts, so M+C normally never starts.
/buy-sell-price	2 eth_call	getMarketConfig + getOracleRate; Refresh repeats both	Functions absent from live selector map.
/faqs	1 REST POST	POST /faq; search after 350 ms; category/page posts immediately	Create/update mutation + one refresh POST; delete one DELETE; Axios has no retry.
other routes	0 application data calls	Header wallet UI / ComingSoon content	Connected wallet/provider discovery remains conditional network traffic.

Admin root cause

The current admin combines Ethereum Sepolia chain configuration, legacy direct-contract functions, and raw-event GraphQL queries with Base deployment values. This is an architecture mismatch, not an internet outage.

Merchant UI route call matrix

Route / trigger	Cold logical calls	What calls	Caveat / repetition
Provider + connected guard	2 subgraph POST on cold success	merchant(id) and merchant(id){channels...} in parallel	Persistent failure max 8 attempts. Dev StrictMode may send/abort/restart both.
Global connected observer	1 balanceOf eth_call	USDC balanceOf(address), mounted across routes	staleTime 0; focus/reconnect/new observer can repeat; failure max 4 attempts.
/	Guard cache + N getChannel	One on-chain detail read per unique nonterminated channel	N same-tick calls normally share one batch POST; stale observers refetch.
/register	1 allowance + 1 balance refetch; 1/2 writes	allowance; optional MAX_UINT approve; registerMerchant; explicit balance refetch	Approve waits for receipt; register resolves after broadcast under current provider nesting.
/account/overview	0/1 shared channels POST + N eth_call	Guard key reused; one getChannel per rendered channel	Fresh cache adds 0 GraphQL; stale remount can add one POST.
/account/payment-channels	0/1 shared POST + N eth_call	Same subgraph key; per-channel on-chain detail	Writes do not invalidate/refetch GraphQL or RPC state, so UI can remain stale.
/account/stake	Global balance only	Deposit/withdraw hooks exist, but current buttons are UI/toast only	No active stake transaction from this page in the snapshot.
/orders	0	Static sample data	No contract/subgraph calls or polling.
QR dialog	1 external image GET	api.qrserver.com only after Accept then Scan	Browser cache-dependent; UPI ID and amount leave the app origin.

Blank/error screen mechanism

The merchant guard requires subgraph profile/channel results. A schema, URL, CORS, or connectivity failure can block rendering and surface 'Failed to fetch merchant data'.

User UI route call matrix

Route / trigger	Cold logical calls	What calls	Caveat / repetition
/	4 token eth_call + 1 config eth_call	useWalletBalance: balanceOf/name/symbol/decimals; getMarketConfig	Normally batched; getMarketConfig is absent live; staleTime 0 refetches.
/buy	1 findBestMerchant eth_call on mount	Broken enabled placement queries (amount=0,type=0); Continue sends createBuyOrder	Can submit a stale merchant; multi-argument UI signature differs from live uint256-only call.
/sell	1 merchant read; then 1 allowance + 1/2 writes	findBestMerchant; allowance; optional approve; createSellOrder	Approval may race create; multi-argument UI signature differs live.
/order?id=	1 immediate + every 6 s	getOrder(orderId) while mounted	10 scheduled reads/min + initial; focus/reconnect add. Missing ID still attempts due enabled bug.
/limits	1 config eth_call + 1 REST POST	getMarketConfig and FAQ POST on mount	FAQ production once; dev StrictMode normally twice because raw effect has no dedupe/cancel.
/transactions	0 current live calls	Order-ID/order helper hooks exist but are dormant	User UI has no subgraph client or active query.
wallet/auth	Session-dependent HTTPS	Embedded login; on FAQ 401: create-auth -> sign -> verify-auth	Original request is not retried; auth 401 can recursively re-enter interceptor.

Wallet-balance message

A failed RPC batch, wrong network, missing client ID, CORS/ad-blocking, or token/chain mismatch can surface 'We could not load your wallet balance'. Four token reads are part of one logical balance query.

User and merchant write choreography

Flow	Logical application calls	Critical behavior
Merchant register	1 allowance read; optional approve write; register write; 1 explicit balance refetch	Approve waits for receipt; register is broadcast-only under current provider nesting; GraphQL is not invalidated.
Merchant add/activate/deactivate	1 Diamond write per click	UI can optimistically change after broadcast, but channel RPC/subgraph caches are not reconciled.
Merchant migrate/terminate	1 migrateAndTerminate(from, ZERO_BYTES32) write	Zero destination can revert under the checked-in source, which requires an approved caller-owned target.
User buy	1 createBuyOrder write	Current hook passes four arguments; live selector accepts only uint256. Navigation uses transaction hash as order ID.
User sell	1 allowance read; optional approve; createSellOrder	Approval may not be mined before create; hook signature differs live; navigation uses transaction hash.
User order actions	1 write each: markPaymentSent, cancelOrder, raiseDispute	raiseDispute UI passes three arguments while live selector accepts bytes32 only.

Implicit wallet and authentication traffic

Trigger	Typical logical calls	Count qualifier
No stored wallet	Local storage inspection	Normally no provider network call.
AutoConnect with saved wallet	Reconnect/provider discovery; 15 s timeout	Multiple buttons/merchant AutoConnect share module/query dedupe.
Connected button details	1 native balance; optional chain metadata; 2 social profile GETs; shared ENS reverse lookup	Cache/session-dependent; these implicit queries disable retry.
Email OTP login	POST send OTP + POST verify OTP	Two embedded-wallet requests plus user input.
Passkey login	GET challenge + POST callback	Two requests plus browser WebAuthn ceremony.
OAuth login	Embedded-wallet login/callback + provider redirects	Variable external traffic.
Injected admin write	Preflight + eth_sendTransaction + block/receipt polling	Physical request count depends on wallet/provider and blocks until inclusion.
Sponsored user/merchant write	Wallet/delegation/paymaster/bundler preflight + send + status/receipt	No defensible fixed HTTP count; record userOp/transaction IDs.

Dormant and zero-call paths

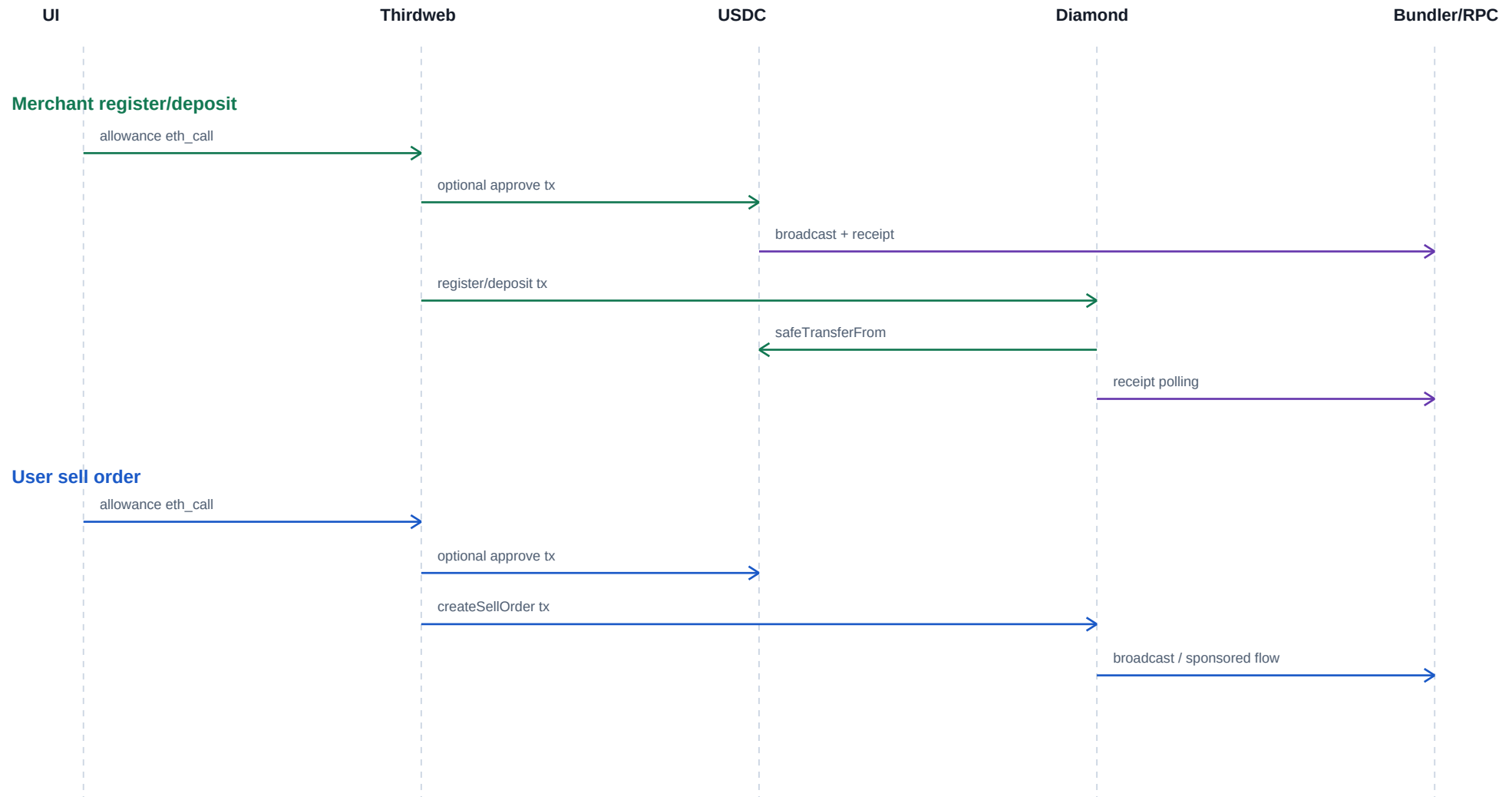
Client	Defined or rendered but not live	Observed network count
User	oracle-rate, my-order-IDs, separate USDC balance, saved UPI/OTP/price/limit helpers; commented withdraw transfer; smart-account/session-key; standalone Thirdweb-auth; PrivateRoutes	0 in current route/provider wiring
Merchant	depositStake, withdrawStake, goOnline, goOffline; edit/migrate popup handlers; legacy GraphQL client; REST layer; activity/platform/meta hooks; smart-account/session-key	0; nominal meta 30 s polling is not mounted
Merchant orders	Static sample rows	0 contract/subgraph calls
Admin	useSubgraphMeta 30 s option, full-config/market/order helpers, duplicate wallet/auth/private-route and legacy transaction components	0 while unreachable/unmounted
Realtime transports	WebSocket and Server-Sent Events	0 across all three UIs

Why include zeroes

A defined hook does not create traffic until a mounted component calls it. The route matrices count observed live consumers, not every exported helper.

Representative transaction sequences

Arrows are logical stages; a stage can produce multiple HTTP/RPC requests.



Caching, retries, and repetition

Mechanism	Observed/default behavior	Effect on counts
TanStack staleTime	User/merchant inner default 0; admin contract reads about 60 s; GraphQL helpers 10 s	Stale mount/focus/reconnect can refetch; helper cache avoids a remount request within 10 s.
Window focus	Default query behavior may refetch stale data; helper subgraph queries disable focus refetch	Returning to the tab can repeat RPC reads but not those helper GraphQL queries.
Retries	Browser default retry=3	Persistent failure can make four attempts for each independent query.
In-flight dedupe	Same query key shares a pending promise	Repeated consumers normally do not multiply a simultaneous request.
RPC batching	Thirdweb same-tick batch, maximum 100 methods	Logical eth_call count can exceed HTTP POST count.
StrictMode development	Mount/effects may be invoked twice	Raw effects may duplicate; query cache usually deduplicates pending same-key reads.
Order timer	setInterval 6000 ms while order ID route is mounted	Ten scheduled refetches per minute, plus the initial query.
Post-write invalidation	Admin can invalidate active reads; user/merchant provider nesting bypasses normal receipt invalidation	Merchant GraphQL/channel views can remain stale; explicit refetches add calls where present.

Failure-mode decision tree

Visible symptom	First discriminating check	Likely layer	Corrective action
Wallet balance cannot load	Does one batched RPC POST reach chain 84532 and return four eth_call results?	Thirdweb/RPC/chain/token config	Validate public client ID, network, token address, CORS/ad blocker, and RPC status.
Merchant data fetch failed	Inspect GraphQL response body/status for the two guard queries	Subgraph URL/schema/indexing	Use active materialized schema; show partial UI and retry control instead of blank gating.
Admin contract read reverts	Compare selector to live loupe	ABI/chain mismatch	Switch to Base 84532 and regenerate exact ABI/hooks.
Admin GraphQL validation error	Compare query root field to schema.graphql	Raw-event vs entity schema mismatch	Rewrite queries for Merchant/PaymentChannel or deploy matching schema.
Write never confirms	Separate wallet/bundler failure from on-chain revert/receipt polling	Thirdweb bundler/RPC/contract	Capture transaction hash/userOp ID, revert data, chain, and sponsorship response.
Data stale after success	Receipt succeeded but GraphQL has not indexed block	Eventual consistency	Poll subgraph head/entity with timeout or optimistic update and reconcile.

Cross-layer mismatch and remediation register

Severity	Area	Evidence / impact	Recommended action
Critical	Source vs live Diamond	Source deploys 5 facets / 36 selectors; live Base Diamond exposes 6 facets / 63 selectors plus a non-routed init ABI entry.	Treat live ABI/storage generation as separate; obtain the exact deployed source and storage layout before upgrading.
Critical	AppStorage upgrade history	Adding nested config.initialized shifted mapping roots from 3/4/5/6 to 4/5/6/7.	Never upgrade across this layout without an explicit migration and verified old layout.
Critical	Admin chain	Admin labels Base but hardcodes Ethereum Sepolia chain 11155111 and ignores VITE_CHAIN_ID.	Point chain, RPC, Diamond, and subgraph to one deployment.
Critical	Admin ABI	Admin calls legacy functions/signatures absent from the live selector map.	Regenerate ABI and adapt each route to the deployed interface.
Critical	Admin GraphQL	UI queries raw event tables; active schema exposes materialized Merchant/PaymentChannel entities.	Rewrite queries to current schema or deploy a compatible subgraph.
Critical	User order signatures	UI uses multi-argument createBuy/createSell/raiseDispute and unavailable market/oracle/matching reads.	Reconcile UI hooks with the exact OrderFacet ABI.
High	Frontend secrets	VITE_THIRDWEB_SECRET_KEY / VITE_APP_SECRET_KEY names are read by browser code.	Anything prefixed VITE_ is public in the bundle; remove server secrets and rotate exposed credentials.
High	Tests vs source	Tests expect DORMANT and functions absent from MerchantFacet.	Bring tests and contract source back to a single protocol generation.
High	Upgrade script	Adds/replaces selectors only, never removes dropped selectors, and supplies no initializer.	Generate full cut diff and explicit migration initializer.
High	Custody escape hatches	ETH/direct ERC-20 can enter the Diamond but current source has no rescue path.	Add governed, audited recovery or prevent unintended transfers.
Medium	Unbounded view	getPendingChannels scans every merchant/channel twice.	Index pending IDs or page results.
High	Query provider nesting	User/merchant Thirdweb hooks use an inner default QueryClient, losing intended 60 s caching/receipt invalidation.	Put one QueryClient at the correct provider boundary and add lifecycle tests.
High	Order navigation ID	User buy/sell navigate with transaction hash, then call getOrder(bytes32) as if it were orderId.	Decode OrderCreated from receipt or return/persist the actual order ID.
High	Merchant post-write sync	Register/channel writes do not invalidate/refetch guard, subgraph, or channel-detail queries.	Wait for receipt, then poll indexer/invalidate exact keys with a bounded timeout.
High	User approval race	User sell approve and create can be broadcast without waiting for approval inclusion.	Wait for approval receipt before creating the order.
Medium	React Query enabled placement	Several hooks place enabled outside queryOptions, so it is ignored.	Move enabled into queryOptions and test disconnected/null-argument states.
Medium	REST configuration	FAQ endpoints and Axios base client use different VITE base names; 401 auth can recursively re-enter.	Use one server base, isolate auth client, add a retry guard, and retry the original request once.
Medium	Indexer gaps	DiamondCut is unhandled; reset timestamps are ignored; missing prerequisite entities can skip later handlers.	Add handler/schema tests, backfill rules, and explicit indexer health metrics.

Recommended convergence sequence

Order	Action	Exit criterion
1	Freeze live Diamond upgrades; export loupe, bytecode metadata, verified source, compiler settings, and exact storage layout.	A reproducible deployed build and storage diff exist.
2	Choose Base Sepolia as one environment and publish one chain/address/subgraph manifest consumed by all UIs.	Admin/user/merchant resolve identical chain ID and Diamond.
3	Generate typed ABI clients from the recovered live facet ABIs; delete legacy hooks.	Every UI signature has a live selector and test.
4	Rewrite admin GraphQL operations to the active materialized schema; add schema validation in CI.	All checked-in queries validate and return representative data.
5	Make guards resilient: distinguish disconnected/loading/empty/error and render retryable UI.	RPC/subgraph outage never produces an unexplained blank page.
6	Move server secrets out of Vite; rotate any browser-exposed credentials.	Bundle scan contains public client IDs only.
7	Add request instrumentation by endpoint class, query key, route, retry number, tx hash/userOp ID.	Observed counts can be reconciled with this report's formulas.
8	Reconcile contract tests with one protocol generation and add storage-layout/selector snapshot tests.	Tests pass against exact source deployed in an ephemeral chain.

Source references

Area	Primary checked-in references
Diamond core	p2pflow-smart-contract/contracts/Diamond.sol; contracts/libraries/LibDiamond.sol
Storage/enums/structs	p2pflow-smart-contract/contracts/shared/AppStorage.sol
Source facets	p2pflow-smart-contract/contracts/facets/*.sol
Initialization/deploy	contracts/upgradeInitializers/DiamondInit.sol; scripts/deploy.js; scripts/upgrade.js
Active deployment ABI	p2pflow-subgraph/abis/Diamond.json
Subgraph topology	p2pflow-subgraph/subgraph.yaml; schema.graphql; src/mapping.ts; src/helpers.ts
Admin calls	p2pflow-admin-ui/src/hooks/useDiamond.js; useSubgraph.js; pages/**
Merchant calls	p2pflow-merchant-ui/src/hooks/useDiamond.js; useSubgraph.js; app/**; pages/**
User calls	p2pflow-user-ui/src/hooks/useDiamond.js; services/blockchain/**; pages/**
Thirdweb internals	Installed thirdweb/viem/TanStack packages in each UI node_modules; defaults summarized, no package source copied.

Reproducibility

The companion Markdown is searchable and contains the same key tables plus Mermaid source diagrams. The generator performs structural PDF validation and asserts the 36/63 function counts.